

If you were a real RTL designer and the Microarchitecture Specification is now signed off, the next step is not RTL coding immediately. The flow becomes:

1. Microarchitecture Review (Critical)

Review with:

- Architecture team

- RTL team

- DV team

- Firmware team

Questions raised:

- Are register definitions complete?

- Any ambiguous behavior?

- What happens on FIFO underflow?

- What happens if CPOL changes during transfer?

- Reset values?

Interrupt priorities?

CDC assumptions?

Many bugs are eliminated here.

2. Create Module-Level Specifications

Break the SPI IP into individual implementation specs.

For example:

Module	Deliverable
APB Interface	APB FSM spec
Register Block	Register implementation spec
Clock Divider	Clock generation spec

TX FIFO	FIFO microarchitecture
RX FIFO	FIFO microarchitecture
SPI FSM	State transition document
Interrupt Controller	Interrupt behavior spec

At this point each block can be owned by different engineers.

3. Draw Detailed FSMs

For example:

text



IDLE

LOAD_SHIFT_REG

ASSERT_CS

GENERATE_CLOCK

SHIFT_DATA

LAST_BIT

DEASSERT_CS

DONE

For every state define:

Inputs

Outputs

Entry condition

Exit condition

This becomes direct RTL guidance.

4. Define Internal Interfaces

Before RTL coding, define signals between blocks.

Example:

text



TX_FIFO

|

| tx_data[31:0]

| tx_valid

| tx_ready

V

SPI_ENGINE

Likewise:

text



SPI_ENGINE

|

| rx_data

| rx_valid

V

RX_FIFO

Without this, RTL integration becomes painful.

5. Create RTL Directory Structure

Example:

text



rtl/

spi_top.sv

spi_apb_if.sv

spi_reg_block.sv

spi_clk_div.sv

spi_tx_fifo.sv

spi_rx_fifo.sv

spi_master_fsm.sv

spi_slave_fsm.sv

spi_interrupt.sv

spi_pkg.sv

6. Create Design Assertions Before Coding

Good designers write basic assertions first.

Example:

systemverilog



```
assert(!(tx_read && tx_empty));
```

systemverilog



```
assert(!(rx_write && rx_full));
```

systemverilog



```
assert(ss_active || !busy);
```

These catch bugs during RTL bringup.

7. RTL Coding Begins

Recommended order:

Week 1

Build:

Package

APB interface

Register block

Verify:

text



CPU writes register

Register updates

CPU reads register

Week 2

Build:

Clock divider

TX FIFO

RX FIFO

Verify:

text



FIFO push

FIFO pop

Overflow

Underflow

Week 3

Build:

Shift register

SPI datapath

Verify:

text



8-bit transfer

16-bit transfer

32-bit transfer

Week 4

Build:

Master FSM

Verify:

text



Mode 0

Mode 1

Mode 2

Mode 3

Week 5

Build:

Interrupt controller

Error handling

Verify:

text



FIFO overflow

FIFO underflow

Mode fault

Transfer complete

8. Designer Unit Testbench

Before handing to DV:

text



tb_apb_if.sv

tb_fifo.sv

tb_clk_div.sv

tb_spi_master.sv

This is not UVM.

Just basic self-checking tests.

Goal:

80% bug removal before DV sees RTL.

9. Static Verification

Run:

Lint

CDC

RDC

Tools commonly used:

Synopsys SpyGlass

Cadence Jasper CDC

Siemens EDA Questa CDC

Fix all critical issues before handoff.

10. Produce Designer Handoff Package

This package goes to DV:

text



RTL

Microarchitecture Spec

Register Specification

FSM Documentation

Assertions

Known Limitations

Unit Test Results

Lint Report

CDC Report

The deliverable after the Microarchitecture Spec that provides the most value is usually the RTL Implementation Plan.

That document contains:

module hierarchy
exact FSM states
signal definitions
coding ownership
milestone schedule
designer test plan

Once that exists, an RTL engineer can start coding with very few open questions.